

Untersuchung und prototypische Evaluation von Self-Healing-Mechanismen in Edge-Kubernetes-Systemen

KIM HEIDELBERG

Bachelorarbeitskolloquium · 28.07.2026

Fakultät Informatik TU Dresden

Institut für Systemarchitektur

Professur für Distributed and Networked Systems

MOTIVATION

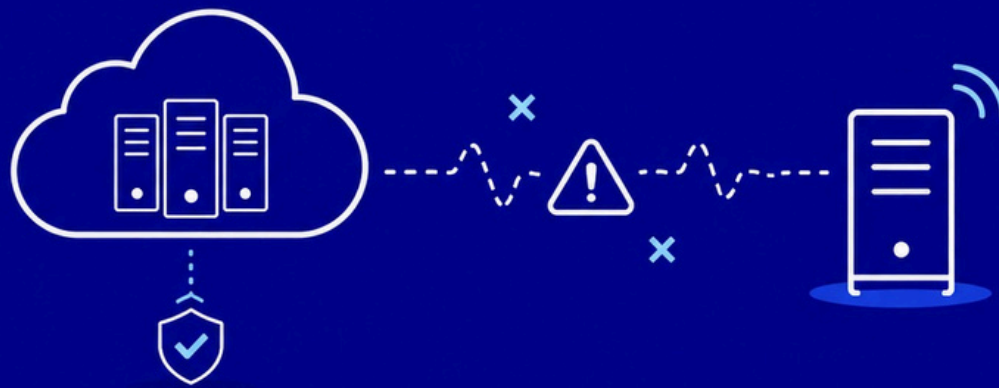


Edge-Anwendungen arbeiten häufig unter instabilen Netzwerkbedingungen. Latenz, Paketverlust und Verbindungsabbrüche gefährden die Service-Verfügbarkeit. Können Kubernetes-Systeme Anwendungen dennoch autonom verfügbar halten?

AGENDA

- Motivation und Forschungsfragen
- Grundlagen und Systemauswahl
- Testumgebung, Versuchsablauf und Methodik
- Ergebnisse
- Fazit, Ausblick und Fragen

FORSCHUNGSFRAGEN



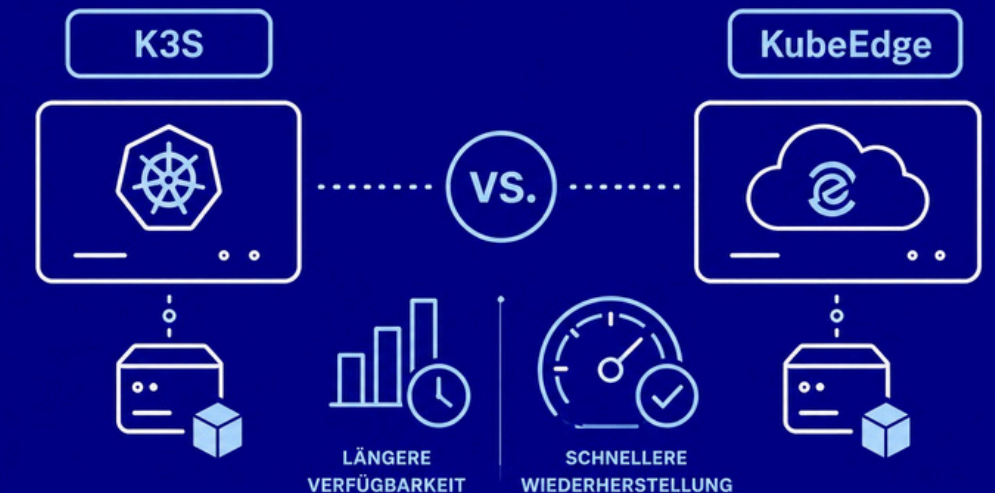
01 Netzwerkstörungen

Wie beeinflussen Latenz, Paketverlust und Verbindungsabbrüche die Service-Verfügbarkeit?



02 Verbindungstrennung

Wie verhalten sich Anwendungen bei einer Trennung zwischen Control Plane und Worker-beziehungsweise Edge-Knoten?



03 Systemvergleich

Welches System hält Services länger verfügbar und stellt schneller einen stabilen Zustand wieder her?

GRUNDLAGEN

Edge Computing

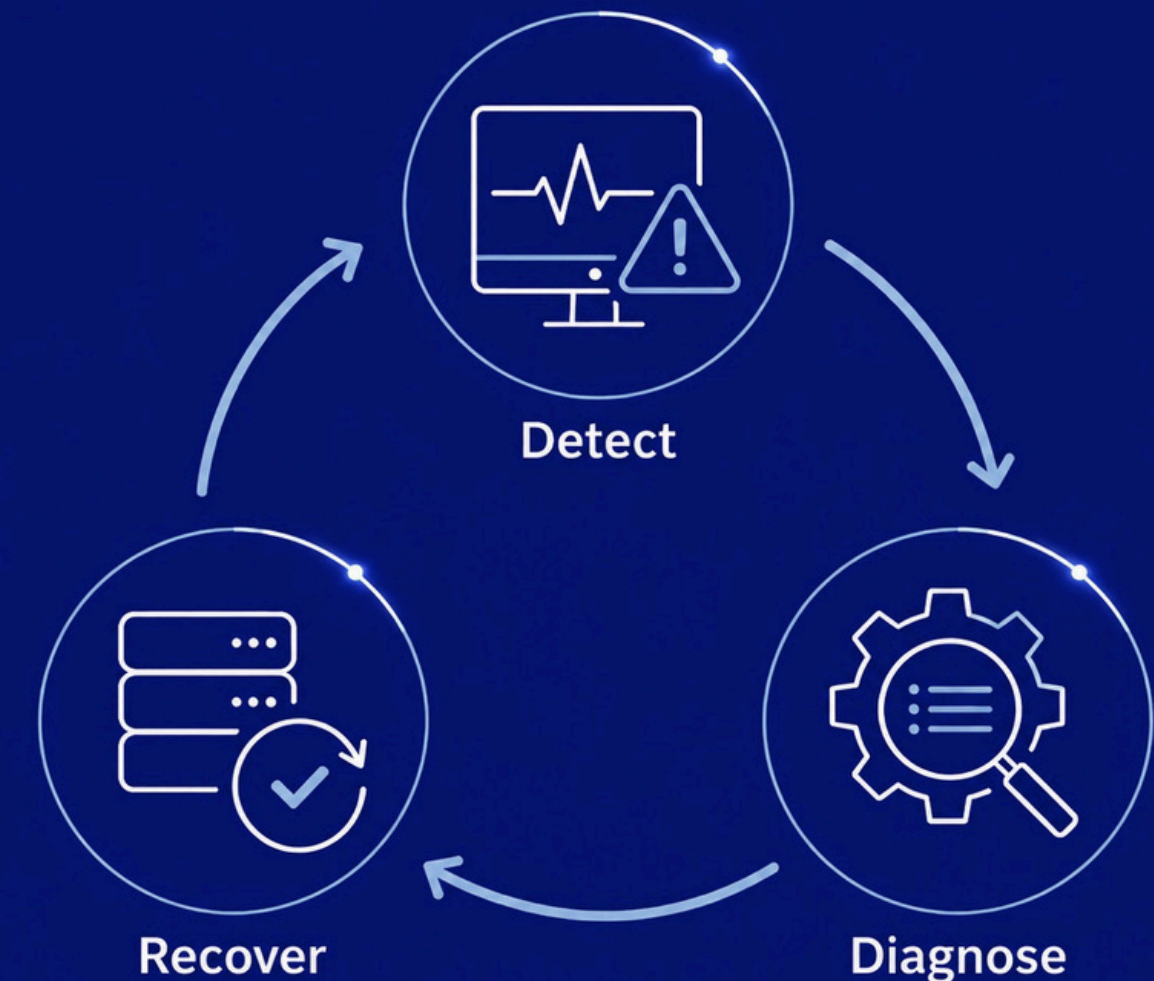
Verarbeitung und Ausführung von Anwendungen nahe an der Datenquelle – häufig unter begrenzten Ressourcen und instabilen Netzwerkbedingungen.

Kubernetes

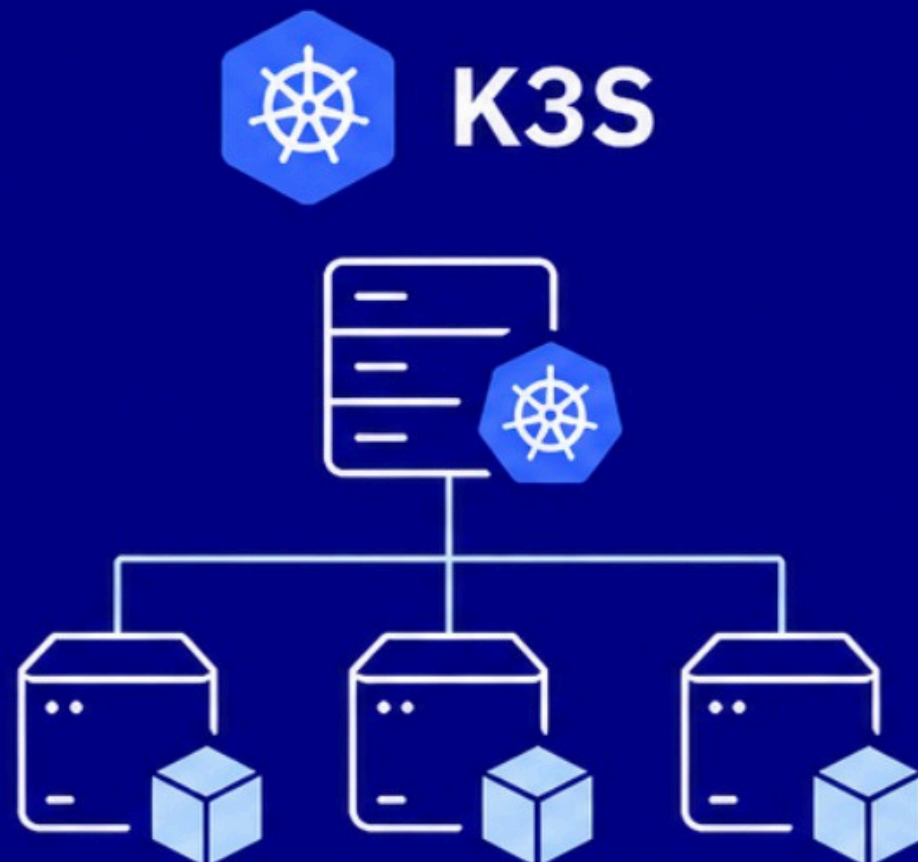
Orchestrierungsplattform, die den gewünschten Zustand containerisierter Anwendungen automatisiert verwaltet.

Self-Healing

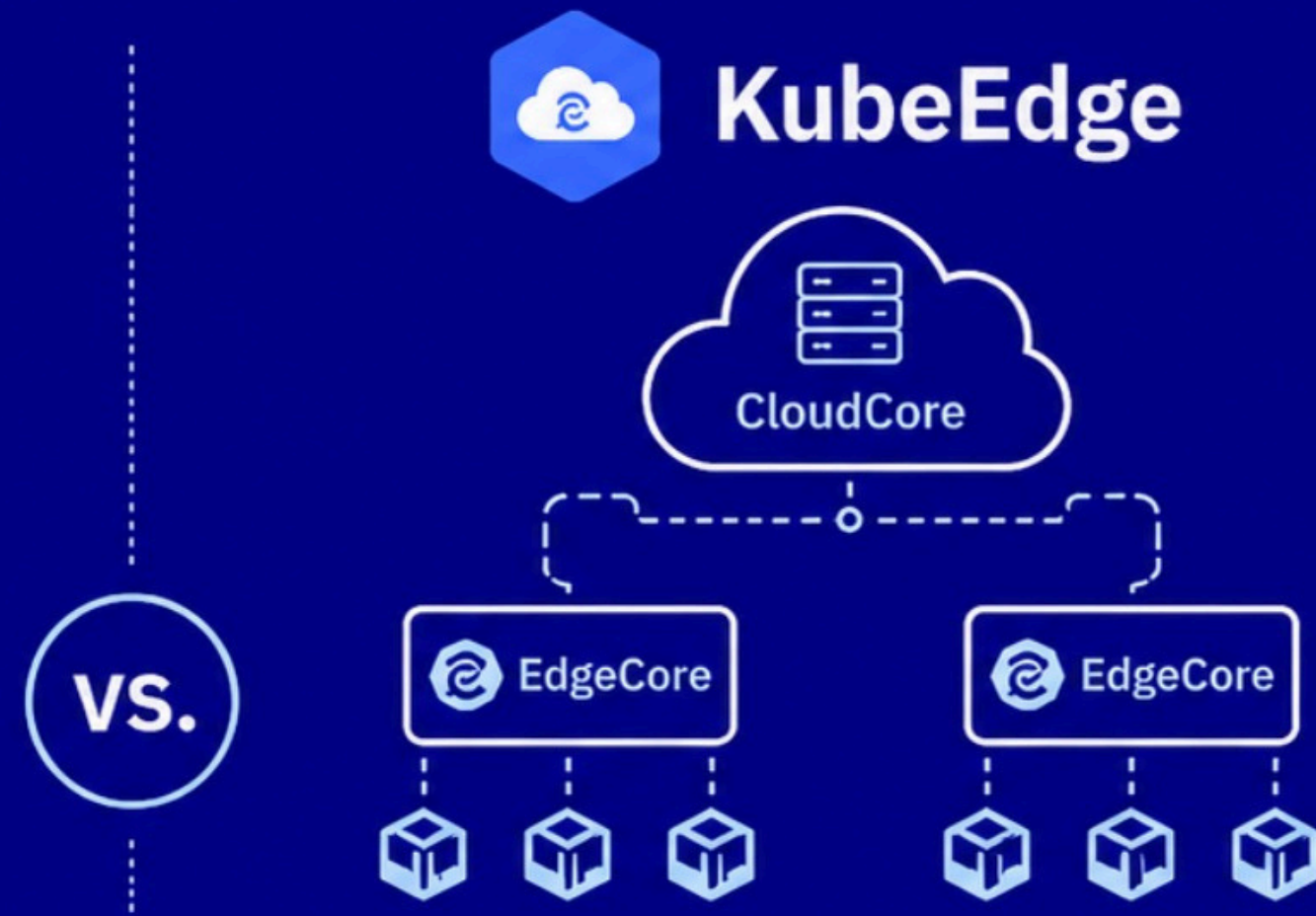
Fehler werden erkannt und durch Neustart, Ersatz oder Neuplatzierung betroffener Workloads automatisch ausgeglichen.



SYSTEMAUSWAHL

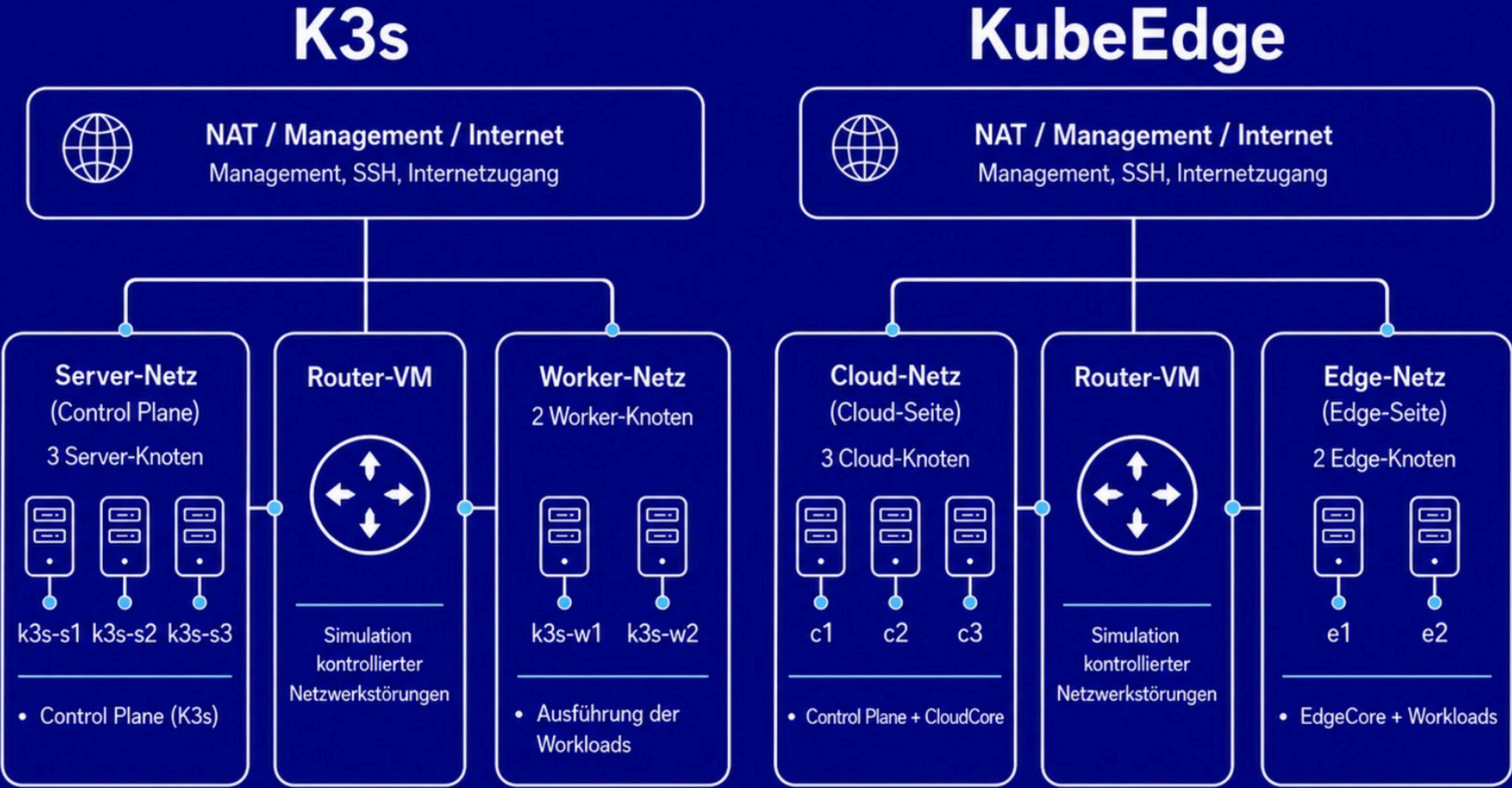


Leichtgewichtige Kubernetes-Distribution
Für ressourcenbeschränkte Umgebungen
Klassische Control-Plane-/Worker-Architektur
Geringere zusätzliche Komplexität

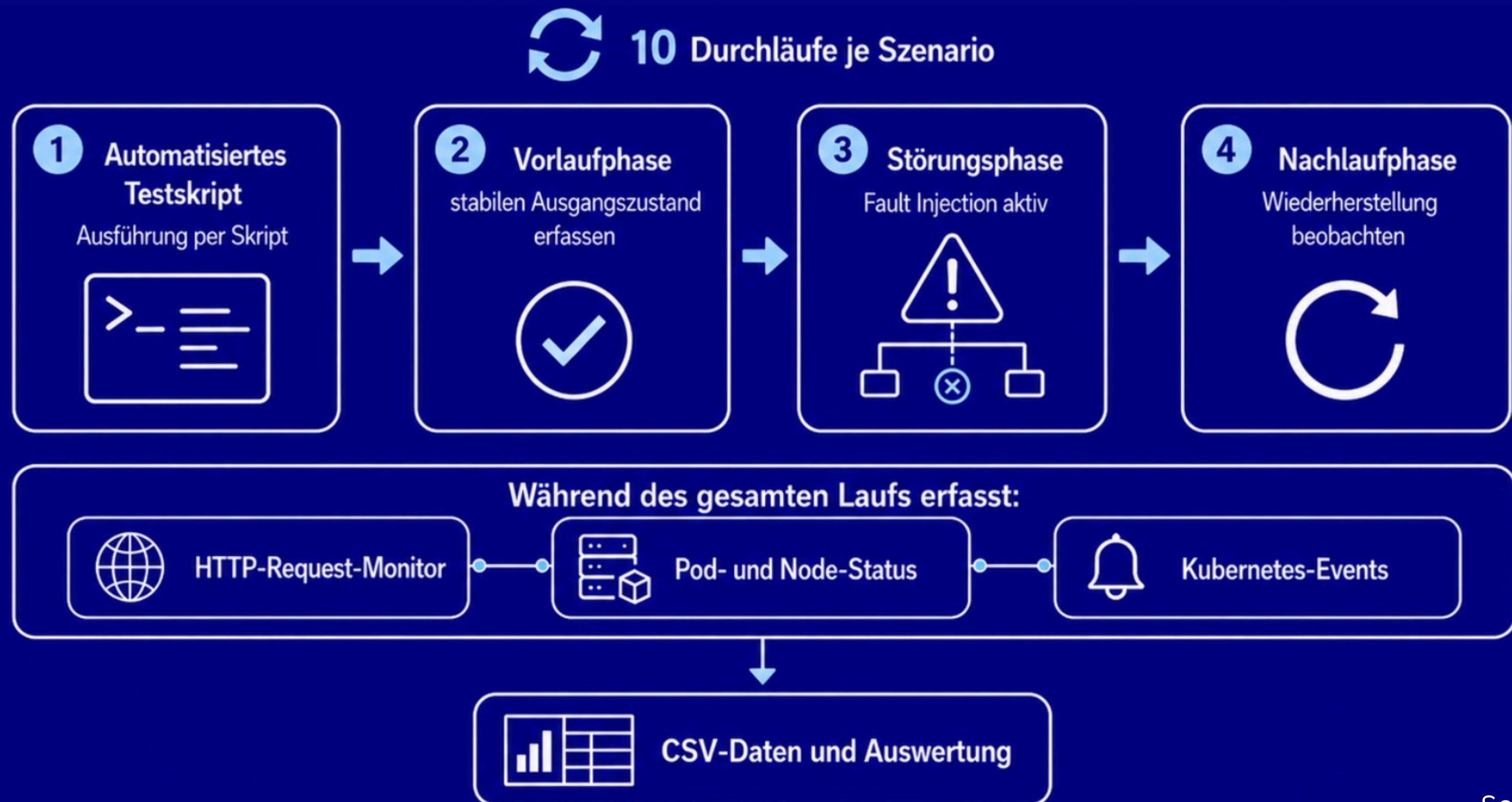


Kubernetes-Erweiterung für Cloud-Edge-Szenarien
Explizite Trennung zwischen Cloud und Edge
Lokale EdgeCore-Komponente
Weiterbetrieb bei temporärer Cloud-Trennung vorgesehen

TESTUMGEBUNG



VERSUCHSABLAUF



METHODIK

Szenario	Parameter
Pod-Ausfall	Pod löschen
Node-Ausfall	Worker/Edge oder Server/Cloud stoppen
Latenz	1s bis 30min
Paketverlust	1% bis 100%
Verbindungsabbruch	1s bis 30min

Metriken
Anwendungsebene

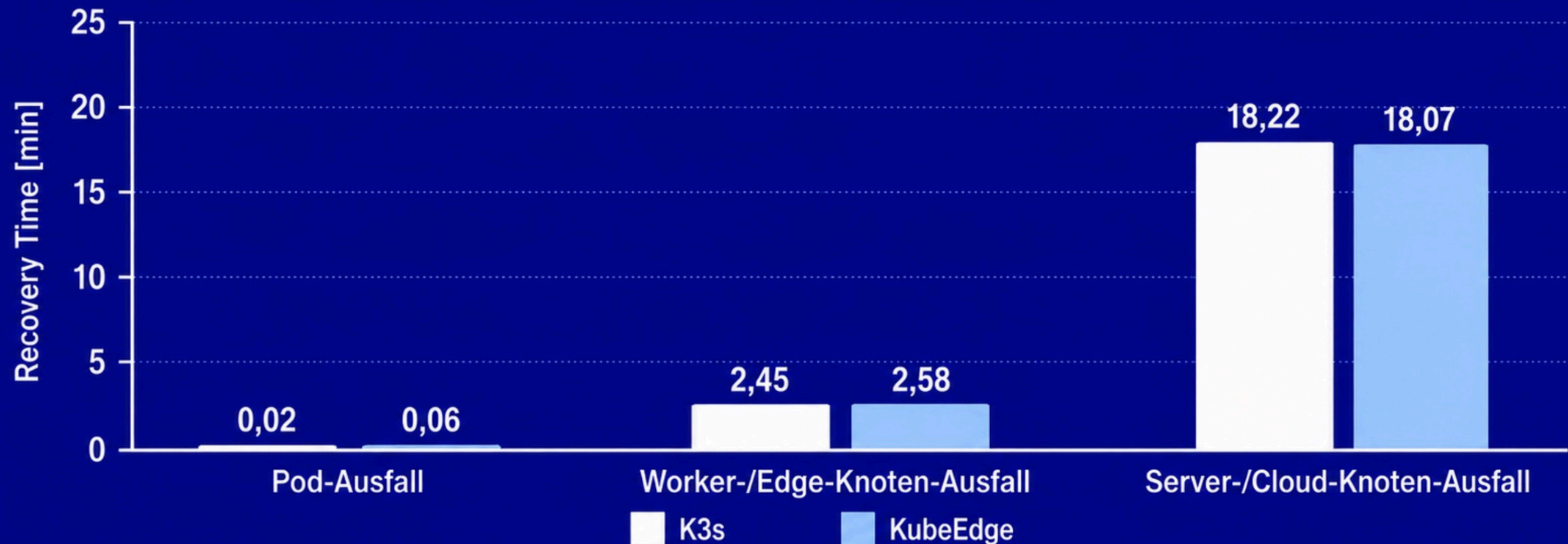
Request Success Rate
Fehlerrate
Recovery Time

Clusterebene

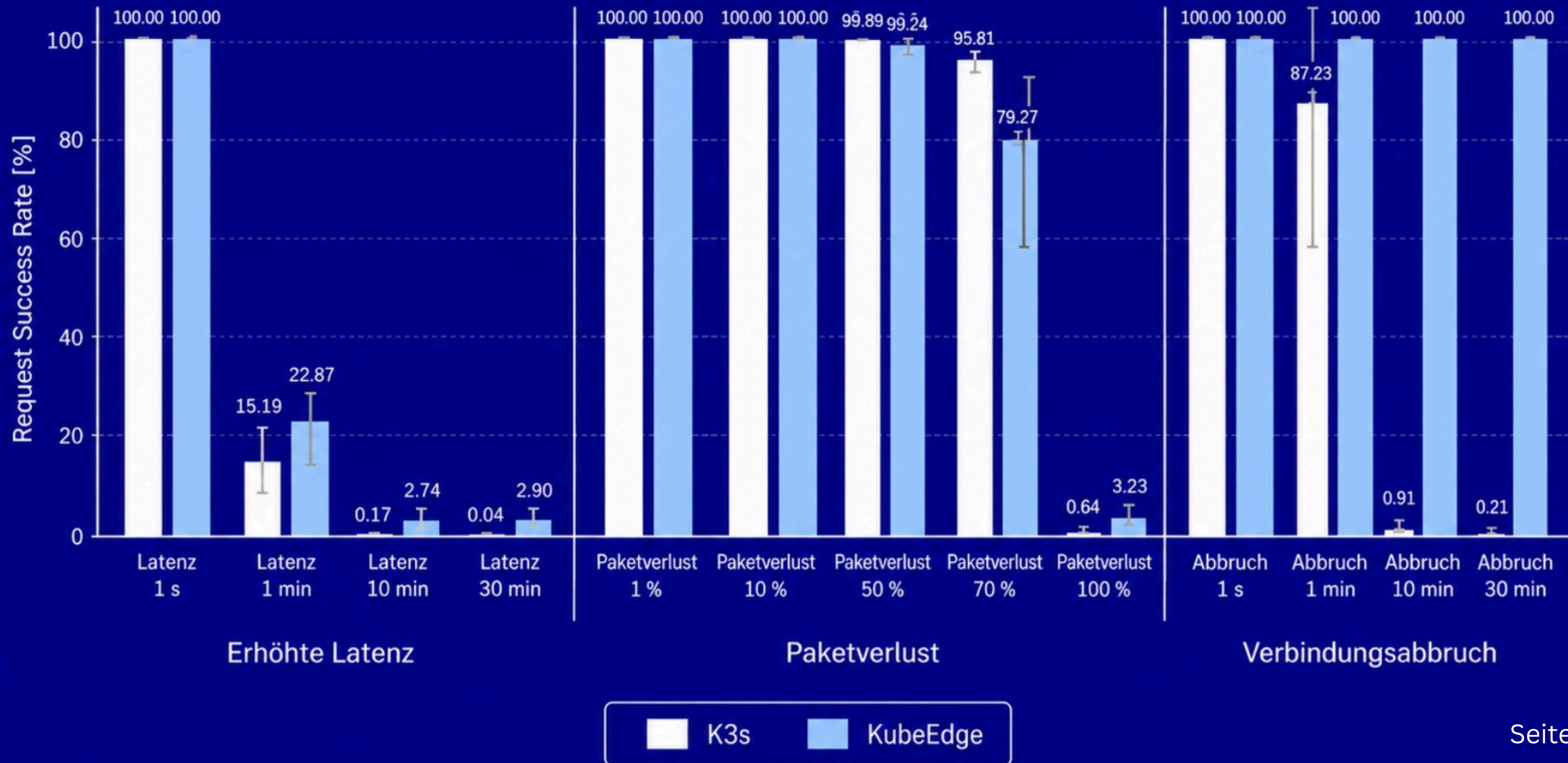
Stabilisierungszeit
Pod-Restarts
Node-Status

POD- UND KNOTENAUSFÄLLE

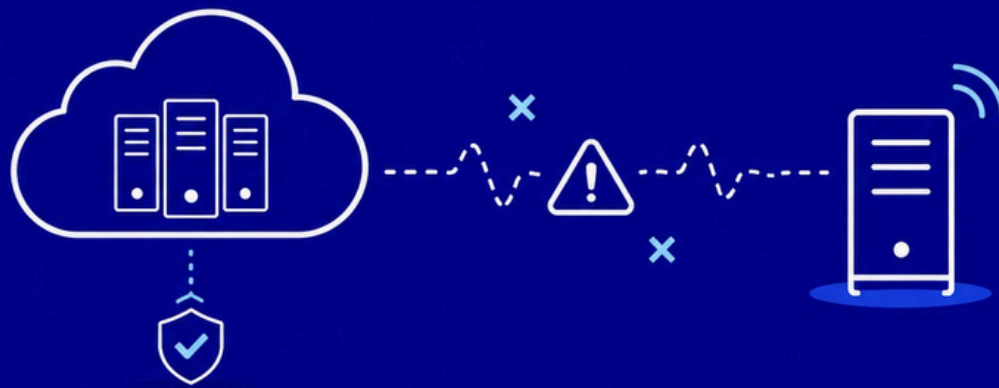
Zeit bis zur Wiederherstellung des stabilen Clusterzustands: Beide Systeme kompensierten Pod-Ausfälle ohne sichtbare Serviceunterbrechung.



NETZWERKSTÖRUNGEN



BEANTWORTUNG DER FORSCHUNGSFRAGEN



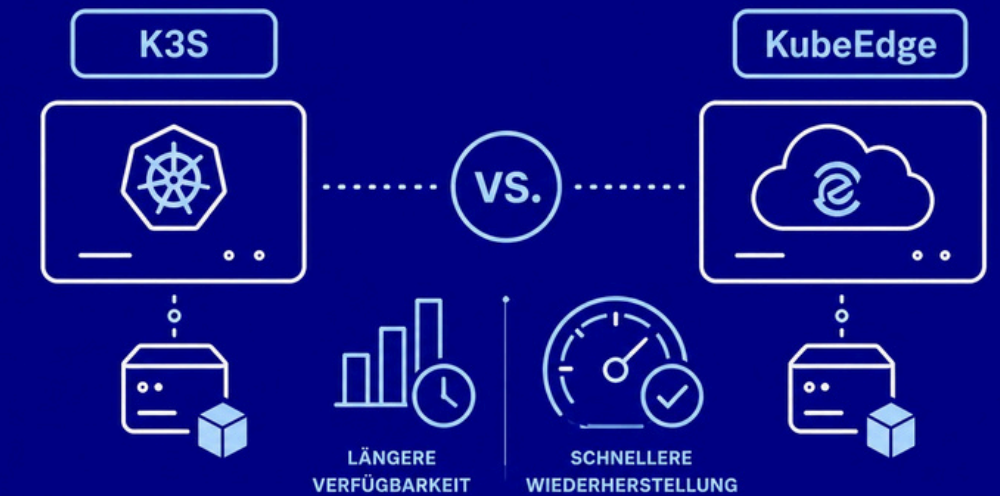
01 Netzwerkstörungen

Geringe Störungen wurden von beiden Systemen toleriert; Aktives Workload-Self-Healing blieb aus.



02 Verbindungstrennung

K3s verlor bei längeren Trennungen weitgehend die Anwendungserreichbarkeit. KubeEdge führte Workloads lokal weiter aus.



03 Systemvergleich

Bei Pod- und Knotenausfällen waren die Systeme ähnlich; bei vollständiger Cloud-Edge-Trennung war KubeEdge klar überlegen

FAZIT

01

K3s

Geeignet für ressourcenbeschränkte Umgebungen mit überwiegend stabiler oder moderat gestörter Verbindung.

02

KubeEdge

Vorteilhaft, wenn Edge-Knoten bei temporärer Cloud-Trennung autonom weiterarbeiten müssen.

Zentrale Erkenntnis

Native Kubernetes-Self-Healing-Mechanismen kompensieren Pod- und Knotenausfälle, reparieren aber gestörte Kommunikationspfade nicht aktiv.

A U S B L I C K

01

Anpassungsbedarf für Space

- Control Plane und Kommunikationsschicht müssen lange Latenzen und intermittierende Verbindungen tolerieren; hierfür sind Store-and-Forward- beziehungsweise DTN-Mechanismen erforderlich
- Zeitkritische Entscheidungen und Fehlerreaktionen müssen lokal und autonom auf den Edge-Knoten erfolgen
- Edge-Knoten müssen Anwendungen über Trennungen von Stunden bis Tagen autonom weiterführen können.

02

Aktuelle Forschungsansätze

Feldmann et al. (WiSEE 2025) [1]: Eine BGP-Erweiterung verteilt automatisch Informationen über die Erreichbarkeit von DTN-Endpunkten. Dies kann die Anbindung IP-basierter Edge-Systeme an delay-tolerante Weltraumnetze vereinfachen

KubeSpace (INFOCOM 2026) [2]: Eine speziell für LEO-Satelliten entwickelte Control Plane mit verteilten Boden-Controllern, nahtlosen Controller-Wechseln und orbitabhängiger Controller-Zuordnung

Danke!

FRAGEN?

Kim Heidelberg

28.07.2026

QUELLEN

[1] A Border Gateway Protocol Extension for Distributing Endpoint Identifier Reachability Information in Delay-tolerant Networks

[Fel+25] Marius Feldmann, Théo Tchilinguirian und Felix Walter. „A Border Gateway Protocol Extension for Distributing Endpoint Identifier Reachability Information in Delay-tolerant Networks“. In: 13th IEEE International Conference on Wireless for Space and Extreme Environments (WiSEE). 2025. arXiv: 2603.01263 [cs.NI]. <https://arxiv.org/pdf/2603.01263>

[2] “KubeSpace: A Low-Latency and Stable Control Plane for LEO Satellite Container Orchestration”

[Zha+26] Zhiyuan Zhao u. a. „KubeSpace: A Low-Latency and Stable Control Plane for LEO Satellite Container Orchestration“. In: IEEE INFOCOM 2026. 2026. arXiv: 2601.21383 [cs.NI]. <https://arxiv.org/pdf/2601.21383>

NACHTRÄGLICHE EINORDNUNG RECOVERY TIME

Szenario	Requests gesamt / fehlgeschlagen	Läufe mit Request-Fehlern	Erster HTTP 200 nach erstem Fehler	Keine weiteren Fehler ab Störungsbeginn
K3s Pod-Ausfall	883 / 0	0 von 10	nicht erforderlich	nicht erforderlich
K3s Worker-Ausfall	2093 / 65	5 von 10	Ø 1,0 s	Ø 57,0 s, Median 54,4 s
K3s Server-Ausfall	11492 / 116	9 von 10	Ø 0,9 s	Ø 65,8 s, Median 53,1 s
KubeEdge Pod-Ausfall	641 / 0	0 von 10	nicht erforderlich	nicht erforderlich
KubeEdge Edge-Ausfall	1926 / 234	10 von 10	Ø 45,2 s	Ø 53,1 s, Median 53,1 s
KubeEdge Cloud-Ausfall	11232 / 0	0 von 10	nicht erforderlich	nicht erforderlich